

Method 1: Bag of Words (BoW) let's understand.

METHOD 1: Bag of Words (BoW)

Basic Idea

In Bag of Words we only look at:

■ Which word appeared how many times.

In this:

- grammar doesn't matter
 - word order doesn't matter
 - only word count is important
-

Real Life Example

Sentence 1:

```
python
"I love cats"
```

Sentence 2:

```
python
"I love dogs"
```

Now vocabulary will be created:

```
python
['i', 'love', 'cats', 'dogs']
```

Now we will count:

Word	Sentence 1	Sentence 2
i	1	1
love	1	1
cats	1	0
dogs	0	1

So vectors will be:

Sentence 1:

python

[1,1,1,0]

Sentence 2:

python

[1,1,0,1]

When do we use BoW?

Best Use Cases

1. Spam Detection

Like:

- "free"
- "win"
- "money"

If they appear repeatedly → it could be spam.

2. Simple Sentiment Analysis

Movie review:

- good
- bad

- awesome

We can make predictions by looking at their counts.

3. Basic Text Classification

Like:

- sports news
 - politics news
 - tech news
-

4. Baseline Model

When starting a Machine Learning project, BoW is tried first.

Why?

- very fast
 - simple
 - easy to debug
-

Problems with BoW

1. Doesn't understand Word Order

Example:

```
python
"I like food"
```

and

```
python
"I do not like food"
```

Both have "like".

But the meaning is different.

BoW gets confused.

2. Doesn't understand Context

Example:

```
python
```

```
bank
```

- river bank
- money bank

BoW treats both as the same.

3. Large Vocabulary Problem

If there are 1 lakh unique words →
the vector will become very large.

Which Package is Best?

Best Package: sklearn

Package:

```
python
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

This is used the most in the industry.

Why?

- simple
 - fast
 - optimized
 - easily fits into ML pipeline
-

Most Important Tool

CountVectorizer

This automatically:

- creates vocabulary
 - does counting
 - creates matrix
-

Simple Example (Industry Style)

```
python

from sklearn.feature_extraction.text import CountVectorizer

# sentences
text = [
    "I love python",
    "I love machine learning"
]

# create object
cv = CountVectorizer()

# fit + transform
result = cv.fit_transform(text)

# vocabulary
print(cv.get_feature_names_out())

# vectors
print(result.toarray())
```

Understand the Output

Vocabulary:

```
python

['learning' 'love' 'machine' 'python']
```

Vectors:

```
python
```

```
[[0 1 0 1]
 [1 1 1 0]]
```

Meaning:

Sentence 1:

```
python
```

```
"I love python"
```

- learning → 0
- love → 1
- machine → 0
- python → 1

Important Interview Point

If interviewer asks:

┆ Why is BoW weak?

Then answer:

- no context understanding
- no semantic meaning
- sparse vectors
- ignores word order

Final Summary

Point	Answer
Main Idea	Word count
Best Package	sklearn
Best Tool	CountVectorizer
Fast?	Yes
Context Understand?	No

Point	Answer
Use in Modern NLP?	Basic/Baseline only
Easy for beginners?	Best

METHOD 2: TF-IDF (Term Frequency - Inverse Document Frequency)

Now let's go one level above BoW.
This is a very important method.
It is widely used in search engines, recommendation systems, and document similarity.

Main Idea of TF-IDF

What was BoW doing?

Just counting.

Problem:

Some words appear everywhere:

- is
- the
- and
- good

These are not useful.

TF-IDF says:

Give more importance to words that are rare and important.

Full Form

Short Form	Meaning
TF	Term Frequency
IDF	Inverse Document Frequency

Simple Understanding

TF = How many times a word appeared in a sentence/document

Formula:

$TF = \text{Word Count in Document} / \text{Total Words in Document}$

IDF = How rare a word is across the entire dataset

$IDF = \log(\text{Total Documents} / \text{Documents containing the Word})$

What does TF-IDF do?

Important Word

If a word:

- appears more in one document
- appears less in other documents

→ Its score will be HIGH.

Common Word

If a word appears everywhere:

- the
- is
- good

→ Its score will be LOW.

Real Example

Documents:

python

1. "I love machine learning"
2. "I love deep learning"
3. "Machine learning is great"

Now:

- "love" appeared in multiple places
- "learning" appeared in multiple places
- "deep" appeared only once

So:

```
python
```

```
deep -> high importance  
love -> low importance
```

When do we use TF-IDF?

1. Search Engine

In systems like Google.

User search:

```
python
```

```
"machine learning"
```

The document where these words are important →
It will appear at the top.

2. Document Similarity

Like:

- resume matching
 - job matching
 - recommendation system
-

3. Text Classification

- spam detection
- topic classification
- sentiment analysis

Gives better results than BoW.

4. Keyword Extraction

For extracting important keywords.

Why is TF-IDF better than BoW?

BoW

TF-IDF

Only count

Gives importance

Common words problem

Reduces weight of common words

Weak representation

Smarter representation

Problems with TF-IDF

1. Doesn't understand Context

Example:

python

bank

- river bank
- money bank

Both the same.

2. Doesn't understand Word Meaning

- car
- automobile

They have the same meaning but will be treated differently.

3. Doesn't understand Word Order

Sentence:

```
python
"not good"
```

and

```
python
"good"
```

Doesn't properly understand the difference.

Which Package is Best?

Best Package = sklearn

Most used:

```
python
from sklearn.feature_extraction.text import TfidfVectorizer
```

Industry standard.

Best Tool

TfidfVectorizer

This automatically:

- creates vocabulary
- calculates TF

- calculates IDF
 - gives the final TF-IDF matrix
-

Simple Example

```
python

from sklearn.feature_extraction.text import TfidfVectorizer

documents = [
    "I love python",
    "I love machine learning"
]

# create object
tfidf = TfidfVectorizer()

# fit + transform
result = tfidf.fit_transform(documents)

# vocabulary
print(tfidf.get_feature_names_out())

# tfidf scores
print(result.toarray())
```

Understand the Output

Vocabulary:

```
python

['learning', 'love', 'machine', 'python']
```

Now values:

```
python

0.57
0.81
```

Such decimals will come.

Why?

Because this is not a count,
it is an importance score.

Real Industry Usage

Resume Screening

HR systems:

- Python
- SQL
- Machine Learning

They extract the importance of these keywords.

Search Systems

Amazon / Google style searching.

Recommendation Systems

Recommending similar articles.

Important Interview Point

If interviewer asks:

■ Why is TF-IDF better than BoW?

Answer:

- reduces importance of common words
 - highlights rare important words
 - better document representation
 - improves text similarity
-

Important Concept

Sparse Matrix

Therefore:

- more memory is needed
 - huge vocabulary problem occurs
-

Final Summary

Point	Answer
Main Idea	Give high score to important words
Best Package	sklearn
Best Tool	TfidfVectorizer
Better than BoW?	Yes
Understands Context?	No
Use in Search Engines?	Very Common
Output	Decimal importance scores

METHOD 3: N-Grams

Now we come to a very important concept:

█ Word groups

This is a very powerful technique in NLP.

Especially in:

- sentiment analysis
- chatbot
- search systems
- next word prediction

it is used a lot.

N-Gram means:

| A group of consecutive words

Types of N-Grams

Type	Meaning
Unigram	1 word
Bigram	2 words
Trigram	3 words
4-gram	4 words

Example

Sentence:

```
python
```

```
"I love machine learning"
```

Unigram

[I], [love], [machine], [learning]

Bigram

[I love], [love machine], [machine learning]

Trigram

[I love machine], [love machine learning]

The biggest problem with BoW and TF-IDF:

┆ They ignore word order.

But in language, order is very important.

Example

Sentence:

```
python
"not good"
```

If we use only unigram:

- not
- good

Then the model can get confused.

But bigram:

```
python
"not good"
```

The whole phrase is understood.

Real Meaning Difference

Sentence	Meaning
good	positive
not good	negative
very good	strong positive

That's why N-Gram captures some context.

When do we use it?

1. Sentiment Analysis

Most famous use.

Because:

- not bad
- very good
- extremely poor

Such phrases are important.

2. Search Suggestions

Google:

python

"how to learn ..."

Next word prediction.

3. Autocomplete

Mobile keyboard suggestions.

4. Spam Detection

Phrases:

- win money
 - click here
 - free offer
-

5. Chatbot / Language Modeling

Next word prediction.

Which Package is Best?

sklearn

The easiest and industry standard.

Package:

```
python  
  
from sklearn.feature_extraction.text import CountVectorizer
```

How to Enable N-Gram?

Using:

```
python  
  
ngram_range
```

Important Settings

Only Unigram

`ngram_range = (1,1)`

Only Bigram

`ngram_range = (2,2)`

Unigram + Bigram Together

`ngram_range = (1,2)`

Simple Example

```
python
```

```
from sklearn.feature_extraction.text import CountVectorizer

text = [
    "not good movie",
    "very good film"
]

# bigram model
cv = CountVectorizer(ngram_range=(2,2))

result = cv.fit_transform(text)

print(cv.get_feature_names_out())

print(result.toarray())
```

Understand the Output

Vocabulary:

```
python

['good film', 'good movie', 'not good', 'very good']
```

Now the model understands the complete phrase.

Unigram vs Bigram

Feature	Unigram	Bigram
Context	Low	Better
Memory	Low	Higher
Accuracy	Medium	Better
Speed	Fast	Slower

Big Problem with N-Grams

Vocabulary becomes very large

If:

- 10,000 words exist

Then:

- possible bigrams will be very many.

Memory increases.

Overfitting Problem

Rare phrases can confuse the model.

Best Practice

Most common:

```
python
ngram_range=(1,2)
```

Why?

Because:

- both unigram + bigram are combined
 - accuracy improves
-

Industry Usage

Task	Common N-Gram
Sentiment Analysis	Bigram
Search	Trigram
Autocomplete	Trigram/4-gram

Task	Common N-Gram
Spam Detection	Bigram
Chatbot	Higher n-grams

Important Interview Point

If interviewer asks:

Why use N-Grams?

Answer:

- captures local context
- preserves word order partially
- improves phrase understanding
- useful for sentiment analysis

Final Summary

Point	Answer
Main Idea	Consecutive word groups
Best Package	sklearn
Main Parameter	ngram_range
Best Use	Sentiment Analysis
Understands Context?	Partially
Memory Usage	High
Most Used	Bigram

METHOD 4: Word2Vec

Now we are moving from Traditional NLP

towards Modern NLP.

Word2Vec was a very big breakthrough in NLP.

What is Word2Vec?

Word2Vec converts words into numbers.

But not simple numbers.

Each word gets a meaningful vector.

Simple Meaning

Suppose:

python

cat

Its vector could be:

[0.12, -0.44, 0.91, 0.33, ...]

And:

python

kitten

Its vector will be close to it.

Why?

Because both have similar meaning.

Main Idea

Similar meaning words → similar vectors

Real Life Analogy

If:

- king

- queen
- man
- woman

The model learns the relationship between these:

king - man + woman $\approx$ queen

This is the famous Word2Vec property.

How does Word2Vec learn?

It reads sentences and:

- ▮ Learns meaning from the words that appear nearby.
-

Example

Sentence:

```
python
"cat drinks milk"
```

Model learns:

- cat is near drinks
- milk is near

So cat becomes related.

Context Window

Word2Vec looks at nearby words.

If window size = 2:

Sentence:

```
python
"I love machine learning a lot"
```

If current word:

python

machine

Then nearby:

- love
- learning

will be considered.

2 Main Types of Word2Vec

Type	Meaning
CBOW	Context → Predict Word
Skip-Gram	Word → Predict Context

1. CBOW (Continuous Bag of Words)

Predicts the missing word by looking at context.

Example:

python

"I ___ python"

Model will say:

python

love

Fast

Good on small datasets

2. Skip-Gram

Predicts the nearby words given the current word.

Input:

```
python  
python
```

Predict:

- love
- coding
- programming

Better embeddings

Powerful on large datasets

Which one is used more?

Model	Use
CBOW	Fast training
Skip-Gram	Better accuracy

Which Package is Best?

gensim

Very famous in the industry.

Package:

```
python  
from gensim.models import Word2Vec
```

Important Parameters

Parameter	Meaning
vector_size	vector length
window	nearby words
min_count	minimum word frequency
epochs	training iterations
sg=0	CBOW
sg=1	SkipGram

Simple Example

```
python

from gensim.models import Word2Vec

sentences = [
    ["i", "love", "python"],
    ["python", "is", "easy"],
    ["i", "love", "machine", "learning"]
]

# train model
model = Word2Vec(
    sentences,
    vector_size=10,
    window=2,
    min_count=1
)

# vector of word
print(model.wv["python"])

# similar words
print(model.wv.most_similar("python"))
```

What will the Output be?

Vector

```
python
```

```
[0.12, -0.44, 0.91, ...]
```

Similar Words

```
python
```

```
[('learning', 0.78), ('easy', 0.65)]
```

Meaning:

Their context was similar.

What is Word Embedding?

Word embedding means:

■ Dense meaningful vector representation

Dense vs Sparse

Type	Example
Sparse	BoW, TF-IDF
Dense	Word2Vec

Example

BoW:

```
python
```

```
[0,0,0,1,0,0,0,0]
```

Mostly zeros.

Word2Vec:

```
python
```

```
[0.23, 0.44, -0.12, 0.91]
```

Compact + meaningful.

When do we use Word2Vec?

Word Similarity

- king $\approx$ queen
 - cat $\approx$ dog
-

Recommendation Systems

Similar products.

Semantic Search

Meaning based search.

NLP Deep Learning Models

To initialize the embeddings layer.

Advantages

Advantage	Meaning
Semantic Meaning	Similar words close
Dense vectors	Memory efficient
Better than TF-IDF	More intelligent
Captures relationships	Powerful

Problems

1. Same word → Same vector

Example:

python

bank

- river bank
- money bank

Both have the same vector.

Context doesn't change.

2. OOV Problem (Out Of Vocabulary)

If a new word comes:

python

chatgptai

Model can fail.

3. Needs Good Dataset

Small dataset → poor embeddings.

Word2Vec vs TF-IDF

TF-IDF

Word2Vec

Frequency based

Meaning based

Sparse

Dense

No semantic meaning

Semantic understanding

TF-IDF	Word2Vec
Simple	Advanced

Industry Reality

Today:

- traditional ML → TF-IDF
- deep learning NLP → embeddings

Word2Vec laid the foundation for modern NLP.

Important Interview Point

If interviewer asks:

Why is Word2Vec better than TF-IDF?

Answer:

- captures semantic similarity
- dense vector representation
- words with similar meaning are closer
- better for deep learning models

Final Summary

Point	Answer
Main Idea	Meaningful word vectors
Best Package	gensim
Dense or Sparse	Dense
Context aware?	No
Semantic meaning?	Yes
OOV Problem	Yes
Famous Models	CBOW, SkipGram

METHOD 5: GloVe (Global Vectors)

Now we come to the next level of Word2Vec.

GloVe also creates word embeddings,
but its learning approach is different.

What is GloVe?

Full Form:

┃ Global Vectors for Word Representation

Main Idea

What was Word2Vec doing?

┃ Learning from nearby context.

But GloVe says:

┃ Learn meaning from which words appear together across the whole dataset.

Simple Understanding

If:

- king often appears with queen
- cat often appears with pet

Then their vectors will be similar.

Main Concept = Co-occurrence Matrix

The most important concept.

What is Co-occurrence?

Which word appeared with which word how many times.

Example

Sentence:

```
python
"I love machine learning"
```

If window size = 1

Then:

Current Word	Nearby Words
love	I, machine
machine	love, learning

Now the model stores these counts.

Co-occurrence Matrix

Suppose vocabulary:

```
python
["cat", "dog", "milk"]
```

Matrix:

Word	cat	dog	milk
cat	0	3	5
dog	3	0	1
milk	5	1	0

Meaning:

- cat and milk appeared together often
- cat and dog also appeared

So vectors will be related.

GloVe vs Word2Vec Main Difference

Word2Vec	GloVe
Local context	Global statistics
Predictive model	Count-based model
Window learning	Co-occurrence matrix

Simple Analogy

Word2Vec

Like:

- Understanding meaning by reading a sentence
-

GloVe

Like:

- Understanding patterns by analyzing the whole book
-

When do we use it?

Pretrained Embeddings

Most common use.

Download already trained vectors and use them.

NLP Deep Learning

- LSTM
- RNN
- Text Classification

Semantic Similarity

Meaning based comparison.

Small Dataset Projects

When your own data is less.

Then pretrained GloVe is very useful.

Famous Pretrained GloVe Models

Released by Stanford.

Popular versions:

Model	Vector Size
glove.6B.50d	50 dimensions
glove.6B.100d	100 dimensions
glove.6B.300d	300 dimensions

Which Package is Best?

Usually:

```
python
gensim
```

Or manually with numpy.

Simple Example

```
python
```

```
import numpy as np

# fake glove embeddings
glove = {
    "cat": np.array([0.2, 0.8]),
    "dog": np.array([0.3, 0.7]),
    "car": np.array([0.9, 0.1])
}

print(glove["cat"])
```

How to calculate Similarity?

Using:

Cosine Similarity

$$\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$$

Meaning

Score	Meaning
1	very similar
0	unrelated
-1	opposite

Simple Similarity Example

```
python
```

```
from numpy import dot
from numpy.linalg import norm
import numpy as np

cat = np.array([0.2, 0.8])
dog = np.array([0.3, 0.7])

similarity = dot(cat, dog) / (norm(cat) * norm(dog))

print(similarity)
```

Why is Pretrained GloVe Powerful?

Because:

- trained on huge internet data
- semantic understanding already exists

Example

If pretrained embeddings are there:

```
python

king ≈ queen
india ≈ delhi
man ≈ woman
```

These relationships are already learned.

Advantages

Advantage	Meaning
Semantic understanding	Similar words closer
Pretrained available	Easy to use
Good embeddings	Better NLP performance
Memory efficient	Dense vectors

Problems

1. Static Embeddings

The biggest problem.

python

bank

Same vector everywhere.

- river bank
- money bank

Doesn't understand the difference.

2. OOV Problem

New word → no vector will be found.

3. Doesn't understand Context

Not as intelligent as BERT.

Word2Vec vs GloVe

Feature	Word2Vec	GloVe
Learning Style	Predictive	Count-based
Uses	Local context	Global statistics
Pretrained Models	Available	Very famous
Performance	Good	Very good

Industry Reality

Today:

- classical deep NLP → GloVe
- modern transformer NLP → BERT

But GloVe is a very important concept for learning embeddings.

Important Interview Point

If interviewer asks:

Why is GloVe sometimes better?

Answer:

- uses global co-occurrence statistics
 - pretrained embeddings available
 - captures semantic relationships well
-

Final Summary

Point	Answer
Main Idea	Global word relationships
Core Concept	Co-occurrence matrix
Best Use	Pretrained embeddings
Semantic meaning	Yes
Context aware	No
Dense vectors	Yes
OOV Problem	Yes

METHOD 6: FastText

Now we come to the method that solves the biggest problem of Word2Vec and GloVe:

Unknown words problem (OOV)

FastText is famous for exactly this problem.

What is FastText?

FastText was created by Facebook AI Research.

It is similar to Word2Vec but:

It doesn't just learn whole words, it also learns small parts of words.

Main Idea

Word2Vec:

```
python  
learning
```

Treats this as one complete word.

But FastText breaks it down:

```
python  
lea  
ear  
arn  
rni  
nin  
ing
```

These are called:

Subwords / Character N-grams

Why is FastText Powerful?

If a new word comes:

```
python
```

learnings

Word2Vec might fail.

But FastText will say:

┆ This has patterns similar to "learning".

So it will generate a vector.

Real Understanding

Suppose:

python
computer

And typo:

python
compter

FastText can still understand.

Why?

Because:

- comp
- ompu
- pute

Such character patterns are similar.

How does FastText work?

Step 1:

Breaks the word into character chunks.

Step 2:

Creates an embedding for each chunk.

Step 3:

Combines all embeddings to create the final word vector.

Example

Word:

python

apple

Possible subwords:

python

app

ppl

ple

appl

pple

Now final vector:

Will be a combination of all these.

FastText's Biggest Strength

OOV (Out Of Vocabulary) Handling

Example

In training data:

python

```
learn
learning
learner
```

Are present.

Now in test:

```
python

learnability
```

Comes up.

FastText can still create a vector.

Which Package is Best?

gensim

```
python

from gensim.models import FastText
```

Important Parameters

Parameter	Meaning
vector_size	vector length
window	nearby words
min_n	minimum subword length
max_n	maximum subword length
epochs	training rounds

What are min_n and max_n?

If:

min_n = 3

And

max_n = 5

Then:

Chunks of 3 to 5 characters will be created.

Simple Example

```
python

from gensim.models import FastText

sentences = [
    ["i", "love", "learning"],
    ["machine", "learning", "is", "fun"]
]

# train model
model = FastText(
    sentences,
    vector_size=10,
    window=2,
    min_count=1,
    min_n=3,
    max_n=5
)

# known word
print(model.wv["learning"])

# unknown/new word
print(model.wv["learnings"])
```

Most Important Thing

See:

```
python

"learnings"
```

Was not in training.

This is the power of FastText.

Word2Vec vs FastText

Feature	Word2Vec	FastText
Word based	Yes	No
Character based	No	Yes
OOV handling	Poor	Excellent
Typos handling	No	Yes
Rare words	Weak	Strong

When do we use it?

In Data with Typos

Social media:

- gmng
 - gud
 - hellooo
-

Morphologically Rich Languages

Like:

- Hindi
- German
- Turkish

Because word forms keep changing.

FastText generalizes better.

Rare Words Problem

Medical/legal domains.

FastText vs GloVe

Feature	GloVe	FastText
Character info	No	Yes
OOV handling	No	Yes
Static embedding	Yes	Yes
Context aware	No	No

Problems with FastText

1. Still Static Embedding

python

bank

Same vector everywhere.

2. Not as Smart as BERT

Doesn't understand sentence context at a deep level.

3. Training is a bit Heavy

Requires slightly more compute than Word2Vec.

Industry Usage

FastText is still widely used today:

- multilingual NLP
 - typo handling
 - search systems
 - low-resource languages
-

Important Interview Point

If interviewer asks:

Why is FastText better than Word2Vec?

Answer:

- uses character-level subwords
 - handles unknown words
 - robust to typos
 - better for rare words
-

Final Summary

Point	Answer
Main Idea	Character/subword embeddings
Best Package	gensim
OOV Handling	Excellent
Typo Handling	Yes
Context aware	No
Better than Word2Vec?	Often yes
Uses Subwords?	Yes

METHOD 7: One-Hot Encoding

This is a very basic and foundational technique in both NLP and Machine Learning.

If in ML you have studied:

- dummy variables
- categorical encoding

Then One-Hot Encoding is a concept from the same family.

What is One-Hot Encoding?

Main Idea:

└ Give each word a unique position.

And:

- Its position = 1
 - Everything else = 0
-

Example

Vocabulary:

```
python
```

```
["cat", "dog", "fish"]
```

Now encoding:

Word	Vector
cat	[1,0,0]
dog	[0,1,0]
fish	[0,0,1]

Why is it called "One-Hot"?

Because:

- Only one place has 1
 - Everything else is 0
-

Simple Visualization

If vocabulary size = 5

python

```
["i", "love", "python", "machine", "learning"]
```

Then:

python

python

Its vector:

[0,0,1,0,0]

How does One-Hot Encoding work?

Step 1:

Create vocabulary.

Step 2:

Give each word an index.

Word	Index
i	0
love	1
python	2

Step 3:

Set that index to 1.

When do we use it?

Small Vocabulary

When words are very few.

Categorical Data

In ML preprocessing:

- city
 - gender
 - color
-

Deep Learning Basics

For understanding the embedding concept.

Educational Purpose

For learning NLP foundations.

Where was it used in NLP?

In old NLP systems.

But used less in modern NLP.

Why?

Because it has big limitations.

Which Package is Best?

sklearn

python

```
from sklearn.preprocessing import OneHotEncoder
```

Easy way in NLP

Can also be built manually.

Simple Example

```
python

vocab = ["cat", "dog", "fish"]

def one_hot(word, vocab):

    vector = [0] * len(vocab)

    index = vocab.index(word)

    vector[index] = 1

    return vector

print(one_hot("dog", vocab))
```

Output

```
python

[0,1,0]
```

sklearn Example

```
python
```

```
from sklearn.preprocessing import OneHotEncoder
import numpy as np

data = np.array([["cat"], ["dog"], ["fish"]])

encoder = OneHotEncoder(sparse_output=False)

result = encoder.fit_transform(data)

print(result)
```

What is a Sparse Vector?

One-Hot vectors are mostly zeros.

Example:

[0,0,0,0,0,0,1,0,0,0]

This is called:

Sparse Vector

Biggest Problem

If vocabulary:

python
100000 words

Then vector length will also be:

python
100000

A lot of memory will be wasted.

Semantic Meaning Problem

The biggest limitation.

Example

```
python  
  
cat = [1,0,0]  
dog = [0,1,0]
```

Model doesn't know:

- cat and dog are similar
 - Both are animals
-

Why is Word2Vec Better?

Because in it:

```
python  
  
cat ≈ dog
```

Vectors are similar.

But in One-Hot:

Everything is completely different.

One-Hot vs Word Embeddings

One-Hot	Embeddings
Sparse	Dense
No meaning	Semantic meaning
Huge vectors	Compact vectors
Simple	Intelligent

Dummy Variable Trap

Important in ML.

If:

- Male
- Female

Two columns are created:

Male	Female
1	0
0	1

Then redundancy can occur.

Therefore:

One column is dropped.

Pandas Relation

Most common:

```
python  
  
pd.get_dummies()
```

This is also One-Hot Encoding.

Example

```
python  
  
import pandas as pd  
  
data = pd.DataFrame({  
    "color": ["red", "blue", "green"]  
})  
  
print(pd.get_dummies(data))
```

Industry Usage Today

Used less directly in NLP today.

But in:

- ML preprocessing
- categorical encoding
- baseline systems

It is still very important.

Important Interview Point

If interviewer asks:

What are disadvantages of One-Hot Encoding?

Answer:

- high dimensionality
- sparse vectors
- no semantic meaning
- memory inefficient

Final Summary

Point	Answer
Main Idea	Unique binary vector
Best Package	sklearn
Vector Type	Sparse
Semantic Meaning	No
Memory Efficient	No
Best For	Small categorical data
Modern NLP Use	Very limited

METHOD 8: Count Vectorization

Now we come to a very practical and industry-used method:

CountVectorizer

Actually:

■ This is the professional/industry version of Bag of Words.

Main Idea

In a sentence:

- Which word appeared?
- How many times did it appear?

Just extract that count.

Example

Sentence 1:

```
python  
"I love python"
```

Sentence 2:

```
python  
"I love machine learning"
```

Vocabulary:

```
python  
['i', 'love', 'machine', 'python', 'learning']
```

Now vectors:

Sentence 1:

[1,1,0,1,0]

Sentence 2:

[1,1,1,0,1]

Difference between Count Vectorizer and BoW?

Bag of Words	CountVectorizer
Concept	sklearn implementation
Can also be manual	Automated
Basic idea	Professional tool

Best Package

sklearn

```
python  
  
from sklearn.feature_extraction.text import CountVectorizer
```

Industry standard.

What does CountVectorizer do Automatically?

Creates vocabulary

Tokenizes text

Lowercase conversion

Word counting

Creates sparse matrix

Most Important Workflow

Step 1

Create object

Step 2

`fit()`

Learns vocabulary.

Step 3

`transform()`

Text → vectors.

Step 4

`fit_transform()`

Both at the same time.

Simple Example

```
python

from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "I love python",
    "I love machine learning"
]

# create object
cv = CountVectorizer()

# fit and transform
result = cv.fit_transform(documents)

# vocabulary
print(cv.get_feature_names_out())

# vectors
print(result.toarray())
```

Understand the Output

Vocabulary:

```
python
```

```
['learning', 'love', 'machine', 'python']
```

Vectors:

```
python
```

```
[[0 1 0 1]  
 [1 1 1 0]]
```

Sparse Matrix

CountVectorizer internally creates a sparse matrix.

Why?

Because:

Most values are zero.

Important Parameters

Now the most important part.

1. stop_words

Removes common useless words.

Example:

- is
- the
- and

Example

```
python
```

```
cv = CountVectorizer(stop_words='english')
```

Benefit

Noise is reduced.

Without Stopwords

Sentence:

```
python
```

```
"This is a good movie"
```

Vocabulary:

```
python
```

```
['this', 'is', 'a', 'good', 'movie']
```

With Stopwords

Vocabulary:

```
python
```

```
['good', 'movie']
```

2. min_df

Removes rare words.

Meaning

Minimum document frequency.

```
python
```

```
min_df = 2
```

Meaning:

Words that did not appear in at least 2 documents →

Remove them.

Why useful?

For removing typos and noisy rare words.

3. max_df

Removes very common words.

Example

```
python
```

```
max_df = 0.9
```

Meaning:

Words that appear in 90% of documents →

Remove.

Why useful?

For removing common useless words.

4. ngram_range

Enables N-grams.

Example

Only unigram:

ngram_range = (1,1)

Unigram + Bigram:

ngram_range = (1,2)

Real Industry Configuration

Most common setup:

```
python

cv = CountVectorizer(
    stop_words='english',
    min_df=2,
    max_df=0.9,
    ngram_range=(1,2)
)
```

When do we use CountVectorizer?

Text Classification

- spam detection
 - fake news
 - sentiment analysis
-

Baseline NLP Models

Feature Engineering

Small/Medium NLP Tasks

Advantage	Meaning
Simple	Easy to use
Fast	Quick training
sklearn integration	Excellent
Good baseline	Very useful

Problems

1. Doesn't understand Context

python
good

vs

python
not good

2. No Semantic Meaning

- car
- automobile

Won't understand they have the same meaning.

3. Sparse Matrix Problem

Large vocabulary → memory issue.

CountVectorizer vs TF-IDF

CountVectorizer	TF-IDF
Raw count	Importance score
Simpler	Smarter
Frequency only	Frequency + rarity

Industry Reality

Still widely used today in traditional ML NLP pipelines with:

- Logistic Regression
- Naive Bayes
- SVM

Important Interview Point

If interviewer asks:

Why use CountVectorizer?

Answer:

- simple text representation
- fast preprocessing
- good baseline
- integrates well with sklearn

Final Summary

Point	Answer
Main Idea	Word frequency counts
Best Package	sklearn
Main Tool	CountVectorizer

Point	Answer
Output	Sparse matrix
Context aware	No
Semantic meaning	No
Best Use	Traditional NLP ML